La récursivité (ou récursion)

I) Introduction

D'une manière générale, la récursivité est la possibilité de faire figurer dans la définition d'une entité, une référence à l'entité elle-même. Elle s'auto-définit. Cette auto-définition peut être directe ou indirecte. Cette notion existe en mathématiques et en informatique.

Exemple - Définition de la factorielle en mathématique :

$0 ! = 1$

$n ! = n * (n - 1) !$ pour tout $n > 0$

Définition

On dit qu'**une fonction est récursive** si elle contient un ou plusieurs appels à elle-même.

Pour certains problèmes, l'utilisation de la récursivité permettra d'obtenir des solutions plus concises et plus simples à coder (*avec un peu d'entraînement toutefois...*).

La théorie est la suivante. Pour écrire l'algorithme qui s'applique à une donnée *D*, on appliquera l'algorithme sur un ou plusieurs sous-ensembles de *D*, jusqu'à ce que le traitement puisse s'effectuer sans nouvelle décomposition.

Les fonctions récursives jouent un rôle très important en informatique. Dès 1936, avant l'apparition des premiers ordinateurs, le mathématicien A. Church a émis la thèse que toute fonction calculable, c'est-à-dire qui peut être résolue selon un algorithme sur une machine, peut être décrite par une fonction récursive. A ce jour, cette thèse n'a pas été contredite.

Exemple : calcul de la factorielle

Version itérative (ou dit autrement version qui utilise une boucle et qui n'est donc pas récursive)

```
/**
 * Renvoie la factorielle d'un nombre entier
 * @param entier entier dont la méthode calcule la factorielle
 * @return un entier égal à la factorielle de l'argument si le calcul est
 *         impossible, à cause d'un argument négatif, la méthode renvoie -1
 */
public static int factorielle(int entier) {
    if (entier < 0) {       // calcul impossible avec entier négatif
        return -1;
    }
    int resultat = 1;           // résultat du calcul
    for (int i = 2; i <= entier; i++) {
        resultat = resultat * i;
    }
    return resultat;
}
```

Remarque : la valeur de la factorielle grandit très vide (Rappel : type *int* sur 4 octets, type long sur 8 *octets*, ainsi que *double*). On obtient les résultats suivants :

factorielle(10) = 3 628 800
factorielle(16) = 2 004 189 184
factorielle(17) = - 288 522 240 => débordement

Comment faire pour aller plus loin dans les calculs ?

On peut utiliser le type *long* au lieu de *int*. Ainsi on pourra calculer :

factorielle(17) = 355 687 428 096 000
factorielle(20) = 2 432 902 008 176 640 000
mais il y aura un débordement avec factorielle de 21

Si on utilise le type *double*, on pourra calculer jusqu'à

factorielle(170) = 7.257 415 615 307 994 E306

Factorielle en récursif

```java
/**
 * Renvoie la factorielle d'un nombre entier
 * @param entier entier dont la méthode calcule la factorielle
 * @return un nombre réel égal à la factorielle de l'argument si le calcul
 *         est impossible, à cause d'un argument négatif, la méthode renvoie
 *         NaN
 */
public static double facto(int entier) {
    if (entier < 0) {       // calcul impossible avec entier négatif
        return Double.NaN;
    }
    if (entier == 0) {      // cas terminal avec entier égal à 1 (condition d'arrêt)
        return 1;
    }
    return entier * facto(entier - 1); // cas général : appel récursif
}
```

Enchaînement des appels:

```
int f = facto(4)
        |→  return  4 * facto(3)
                      |→  return 3 * facto(2)
                                    |→  return 2 * facto(1)
                                                  |→  return 1 * facto(0)
                                                                |→  return 1
                                                                (cas terminal)
```

Condition d'arrêt

Pour être calculable, les définitions récursives ont besoin d'une condition d'arrêt (ou cas terminal). Dans notre exemple, c'est `entier == 0`. Au fil des appels récursifs, le paramètre de la fonction (ou les paramètres) doit évoluer pour restreindre le domaine d'application et tendre vers le cas terminal qui permettra d'arrêter la succession des appels récursifs.

Ici la valeur de l'argument *entier* décroit à chaque appel récursif, pour tendre vers 0, le cas terminal.

Récursivité croisée

On parle de récursivité croisée lorsqu'une fonction *f1* appelle une fonction *f2*, et que *f2* appelle elle-même *f1*.

II) Comparaison fonction récursive et itérative

a) Inconvénient de la récursivité : perte d'efficacité

- ✓ en temps d'exécution, car les appels aux fonctions sont nombreux et coûteux en temps d'exécution
- ✓ et aussi en occupation mémoire : les paramètres sont empilés, et aussi les variables locales si il y en a. C'est pourquoi, lorsque c'est possible, on évitera de déclarer des variables locales dans une fonction récursive.

Exemple d'amélioration de l'efficacité de la fonction factorielle

On remarque le test du cas où le calcul est impossible `if (entier < 0) ...` est effectué à chaque appel.

Pour éviter la réalisation de ce test à chaque appel récursif, il est possible d'utiliser la notion de **méthode (ou fonction) enveloppe.** Il s'agit d'une fonction non récursive qui effectue des tests, des opérations préalables, et ensuite appelle la fonction récursive. Cette dernière sera définie comme une fonction privée.

```
/**
 * Renvoie la factorielle d'un nombre entier (méthode enveloppe)
 * @param entier entier dont la méthode calcule la factorielle
 * @return un double égal à la factorielle de l'argument. Si le calcul
 *         est impossible, à cause d'un argument négatif, la méthode renvoie
 *         NaN
 */
public static double factorielle(int entier) {
    if (entier < 0) {       // calcul impossible avec entier négatif
        return Double.NaN;
    }
  return facto(entier);
}

/**
 * Renvoie la factorielle d'un nombre entier
 * @param entier entier dont la méthode calcule la factorielle
 *               il doit être positif ou nul
 * @return un nombre réel égal à la factorielle de l'argument
 */
private static double facto(int entier) {
    if (entier == 0) {      // cas terminal avec entier égal à 0 (condition d'arrêt)
        return 1;
    }
    return entier * facto(entier - 1);     // cas général
}
```

Illustration de l'occupation mémoire avec la pile d'exécution

Supposons que la définition de la fonction récursive soit maintenant :

```
/**
 * Renvoie la factorielle d'un nombre entier
 * @param entier entier dont la méthode calcule la factorielle
 *               il doit être positif ou nul
 * @return un nombre réel égal à la factorielle de l'argument
 */
private static double facto(int entier) {
    double resultat;
    if (entier == 0) {      // cas terminal avec entier égal à 1 (condition d'arrêt)
        resultat = 1;
    } else {
      resultat = entier * facto(entier - 1);
  }
    return resultat;        // cas général
}
```

On constate que l'on a ajouté une variable locale, resultat, qui n'est pas indispensable. La présence de cette variable va nécessiter plus d'espace mémoire pour chacun des appels récursifs.

Supposons que l'on invoque la fonction de la manière suivante : `int f = facto(3);`
Schématiquement, la pile d'exécution suivra l'évolution suivante :

		resultat
4ème appel : facto(0)	0	*entier*
		resultat
3ème appel : facto(1)	1	*entier*
		resultat
2ème appel : facto(2)	2	*entier*
		resultat
1er appel : facto(3)	3	*entier*
		f

Remarque : En toute rigueur, sont empilés aussi des variables temporaires pour effectuer des calculs tels que *n - 1* et la multiplication, et également les adresses de retour pour chaque appel.

Au fur et à mesure que les calculs sont faits, les variables empilées sont dépilées. On constate que l'on a intérêt à éviter les déclarations de variables locales qui ne sont pas indispensables. Les appels récursifs étant nombreux, le risque est grand de saturer la pile. Il faut donc veiller qu'à chaque appel le minimum d'espace mémoire soit consommé.

b) Avantages de la récursivité :

- ✓ facilité d'écriture. Sur l'exemple donné ce n'est pas flagrant, mais il existe des algorithmes très simples à écrire dans leur forme récursive et très complexes en itératif (voir exemple en TP). On l'utilise pour écrire des traitements sur des structures de données récursives telles que les arbres ou les graphes, pour faire du prototypage ... (les arbres interviennent notamment dans les stratégies de jeux)

- ✓ des langages de programmation sont uniquement récursifs : par exemple des langages fonctionnels tels que Lisp ou Caml ou le langage logique Prolog.

Un algorithme récursif peut toujours être remplacé par son équivalent itératif, mais parfois au détriment de la facilité d'écriture de l'algorithme. Dans certains cas, essentiellement lorsque les structures de données à manipuler sont elles-mêmes récursives ou bien pour des problèmes mathématiques définis par récurrence, la version itérative sera très complexe à écrire, et peu lisible. Au contraire, la version récursive sera simple à coder et lisible.

Donc pour certains algorithmes naturellement récursifs, l'écriture de l'algorithme récursif sera plus simple que son équivalent itératif *(avec peut-être un peu d'entraînement, toutefois...)*

c) Principe d'écriture – Conseils à suivre

- ✓ Ecrire l'en-tête de la méthode. Attention : parfois il faut ajouter un paramètre qui n'existerait pas dans la version itérative
- ✓ Rechercher un (ou plusieurs) cas terminal : attention, il doit y avoir au moins un cas terminal
- ✓ Dans le cas général (il peut y en avoir plusieurs), il faut faire "décroitre la taille" du problème à résoudre. Il faut se rapprocher du cas terminal, tendre vers le cas terminal.

Exemple de fonction récursive qui s'applique à un tableau (méthode enveloppe)

```
/**
 * Cette méthode détermine le nombre d'apparitions d'une valeur dans un
 * tableau d'entiers. Cette méthode est une méthode "enveloppe"
 * qui invoque une méthode récursive
 * @param t  le tableau dans lequel chercher la valeur à compter
 * @param valeur valeur dont on compte les occurrences
 * @return int nombre d'occurrences trouvées
 */
public static int compterOccurrence(int[] t, int valeur) {
    return compterOccurrence(t, valeur, 0);
}



/**
 * Cette méthode détermine le nombre d'apparitions d'une valeur dans un
 * tableau d'entiers.
 * @param t  le tableau dans lequel chercher la valeur à compter
 * @param valeur valeur à comptabiliser
 * @param debut  indice à partir duquel il faut effectuer la recherche
 * @return int nombre d'occurrences trouvées
 */
private static int compterOccurrence(int[] t, int valeur, int debut) {
    if (debut == t.length) {        // le tableau a été entièrement parcouru
        return 0;                   // il n'y a plus d'occurrences
    }
    if (t[debut] == valeur) {

        /*
         * la case courante contient la valeur recherchée : le nombre total
         * d'occurrences sera égal à 1 + le nombre d'occurrences trouvées
         * à partir de l'indice debut + 1
         */
        return 1 + compterOccurrence(t, valeur, debut+1);
    }

    /*
```

```
     * sinon : La case courante ne contient pas la valeur recherchée :
     * le nombre d'occurrences est égal au nombre d'occurrences trouvées
     * à partir de l'indice debut + 1
     */
    return compterOccurrence(t, valeur, debut + 1);
}
```

Suggestion d'exercice

Utiliser la fonction ***System.nanoTime()*** pour tester et comparer l'efficacité d'une même fonction écrite en récursif et en itératif.

III) La récursivité dans les langages de programmation

Certains langages ou certaines versions de certains langages ne gèrent pas la récursivité : Cobol, Fortran, Basic ...

D'autres langages sont au contraire battis sur la récursivité. Quelques exemples de tels langages :

Langage Lisp

- ✓ Lisp = list processing : tous les traitements sont basés sur des listes
- ✓ impératif et fonctionnel, créé en 1958, a beaucoup évolué depuis
- ✓ en 1994, naissance de CommonLisp, qui fait l'objet d'une norme ANSI. Le but était d'unifier plusieurs formes de Lisp qui posaient problème de part leur diversité.
- ✓ Dans les années 70 et 80, Lisp a beaucoup été utilisé en intelligence artificielle, a servi à écrire un éditeur de textes Emacs. Utilisé aussi pour écrire du code dans la distribution Debian de Linux
- ✓ Actuellement :
  - les chercheurs l'utilisent toujours en intelligence artificielle
  - sert aussi à écrire des systèmes experts, faire du prototypage ou du développement rapide, de la conception en 3D
  - c'est un langage intégré au logiciel AutoCAD (l'un des logiciels les plus utilisés pour la CAO)

Langage Caml

- ✓ né en 1985, style fonctionnel, version orientée objet OCaml, utilisé pour l'enseignement

Langage Scheme (prononcer Skim)

- ✓ né en 1975, peut être vu comme un dialecte de Lisp

Langage Prolog

- ✓ Prolog = Programmer en logique, en 1971
- ✓ utilisé en intelligence artificielle (systèmes experts) et pour les traitements portant sur la linguistique
- ✓ basé sur l'écriture de prédicats
- ✓ a donné naissance à la programmation logique par contrainte, utilisée pour les systèmes experts (plusieurs langages descendants de Prolog, et Oz (né en 1995), par exemple)